DXエンジニア向け｜Claude Code 工程別チートシート

8工程＋使用例集＋シーン別（基本・応用・ニッチ）＋Docker/MCP/エラー/環境構築/Skill/サブエージェント＋戻す・差分

1 現状分析

2 目的

3 ゴール

4 範囲

5 回収

6 実装

7 テスト

8 レビュー

使い方：VSCodeの統合ターミナル（Ctrl+`）で claude を起動。使用例のプロンプトは日本語のまま貼れます。確認日 2026/10/18

PHASE 1–2

①現状分析 ／ ②目的

まず読むだけ。次に「何のためにやるか」を言葉にする

1 現状分析

プランモードで読み、事実・推測・不明に分ける

`claude --permission-mode plan`	読むだけで起動
`Shift+Tab`	plan mode on に切替
`/context`	記憶の使用量を確認
`code-explorer など`	調査を別室（サブエージェント）へ

使用例

サブエージェントを使って、請求バッチの
処理の流れと呼び出し元を調べて。
git履歴も見て経緯を要約し、確認できたこと／
推測／不明の3つに分けて。

2 目的

なぜ引き継ぐのか・何を守るのかを確定

`/plan 依頼`	計画だけ立てさせる
インタビュー方式	わからない点だけ質問させる
`SPEC.md`	目的を1段落で残す

使用例

この改修の目的を整理したいので、私に
インタビューして。コードから読めることは
先に調べ、業務上の理由だけ質問して。
結果は SPEC.md の「目的」に書き出して。

PHASE 3-4

③ゴール ／ ④範囲

「どうなれば完了か」と「どこまでやるか」を先に決める

3 ゴール

完了条件を、合否が出る形で書く

`/goal 条件`	条件を満たすまで継続
結果を表示	条件に必ず入れる
`/run /verify`	実際に動かして確認

使用例

/goal `npm test` の結果が会話に表示され、
全テストが成功していること。
加えて、請求バッチの出力CSVが旧システムの
出力と一致することを表示して確認すること。

4 範囲

対象と「やらないこと」を名指しで書く

`@ファイル名`	対象ファイルを指定
`SPEC.md「やらないこと」`	範囲外を明記
範囲を絞る	investigate を無制限にしない

使用例

対象は @src/billing/ のみ。DBスキーマと
他モジュールは変更しないで。
範囲外を直したくなったら、変更せずに
SPEC.md の「やらないこと」へ追記して。

PHASE 5–6

⑤回収 ／ ⑥実装

分かったことを文書に残してから、小さく作る

5 回収

会話ではなく文書・スキル・フックに残す

`/init`	CLAUDE.md の土台
`/memory`	共通ルールを回収
`/doctor`	CLAUDE.md を削る提案
`/hooks`	毎回必ず動かす処理

使用例

ここまでの調査で、毎回守るべきルール（テスト実行方法・命名・禁止事項）だけを CLAUDE.md に追記して。コードから分かることは書かない。
終わったら /clear します。

6 実装

計画→小さく実装。隔離して並列に

`/plan 依頼`	実装前に計画を作る
`claude -w 名前`	専用worktreeで作業
`/simplify`	整理（バグ探しではない）
`/batch 指示`	大量ファイルへ同じ変更

使用例

SPEC.md を読んで、請求バッチのリファクタ計画を /plan で作って。
承認したら worktree 内で小さな単位に分けて実装し、各単位でテストを実行して。

PHASE 7–8

⑦テスト ／ ⑧レビュー

失敗するテストが先。レビューは書いた人と別の目で

7 テスト

現行動作を固定し、通るまで回す

`/goal 条件`	全テスト成功まで継続
`/verify`	ビルド・起動して観察
`test-writer（worktree）`	現状のテストを書く係
証拠を見せて	出力とコマンドを提示

使用例

test-writer で、請求バッチの現行動作を固定する
テストを書いて。本番コードは変更しない。
まず再現する失敗テストを書き、それから直して。
テスト出力と実行コマンドを見せて。

8 レビュー

目的別に使い分け、不足だけ拾う

`/code-review high --fix`	正しさのバグ（--comment でPRへ）
`/security-review`	インジェクション・認可・露出
別セッション／`reviewer`	書いた人と分ける

使用例

サブエージェントで、この差分を PLAN.md と
照らしてレビューして。要件の実装漏れ、
エッジケースのテスト、範囲外の変更だけ報告。
スタイルの好みは不要。

ADD-ON 1

Skill作成

毎回同じ手順を説明しているなら、スキルにする。置き場所は .claude/skills/名前/SKILL.md

SKILL.md のひな形

```
---
name: billing-test-planner
description: 請求バッチのテスト観点を洗い出し、
  テスト計画を作るときに使う。
---
# 手順
1. SPEC.md の「目的」「範囲」を読む
2. 正常系・境界値・異常系を一覧にする
3. 現行動作を固定するテストから書く
4. テスト出力を証拠として提示する

# やらないこと
- 本番コードは変更しない
```

どこに残す？（回収先の判断）

- 常に守るルール → CLAUDE.md
- たまに使う手順・ドメイン知識 → スキル
- 毎回必ず実行したい処理 → フック（/hooks）

作成を頼む例（そのまま貼れる）

- 「今日の請求バッチ調査の手順を、スキルにして。.claude/skills/ に SKILL.md で作って」
- description は「いつ使うか」を具体的に

関連コマンド・注意

- /run-skill-generator：/run と /verify に起動方法を教えるスキルを作る
- サブエージェントの skills 項目で事前読み込みも可
- 詳細ドキュメントはリンクで足りる。短く書く

ADD-ON 2

タスク分解・進め方

「一気にやらせない」。計画→分解→単位ごとに実装→検証、を繰り返す

1 **SPEC.md**
目的・範囲・やらないこと

2 **/plan**
実装前に計画を作る

3 **PLAN.md**
小さな単位に分解して保存

4 **単位ごとに実装**
worktreeで隔離

5 **/goal**
単位ごとに合格条件

6 **レビュー**
PLAN.mdと照合

使用例（分解を頼む）

SPEC.md を読んで、請求バッチのリファクタを
独立して実装・テストできる小さな単位に分解して。
各単位に、①対象ファイル ②完了条件（実行できるテスト）
③依存する単位 を書いて、PLAN.md に保存して。
実装はまだ始めないで。

承認後：「単位1だけ実装して。終わったら
テストの実行結果を見せて」

分解・並列で使うコマンド

`/tasks`	動いているタスクの確認
`/subtask <task>`	脇の作業→完了で会話に戻る
`/batch <指示>`	独立した5〜30単位に自動分割
`claude agents`	複数セッションを一画面で
`Ctrl+B`	実行中を背景へ回す

ADD-ON 3

テスト立案

専用の「テスト立案コマンド」はありません。計画系コマンド＋依頼文＋test-writerで進めます

① 観点を出す（plan mode）

SPEC.md と @src/billing/ を読んで、
テスト観点を一覧にして。正常系／境界値／
異常系／業務ルール別に分け、根拠の
ファイルと行を付けて。まだテストは
書かないで。

② 計画にする（/plan）

観点を、優先度つきのテスト計画にして。
現行動作の固定を最優先に。各テストの
合格条件（実行コマンドと期待結果）を書き、
PLAN.md の「テスト」に保存して。

③ 書かせる（test-writer）

test-writer で、PLAN.md の優先度1〜3を
書いて。モックは使わない。本番コードは
変更しない。まず失敗するテストを書き、
実行結果を見せて。

使うコマンド

`/plan 依頼`	計画だけ立てる
`@ファイル`	対象を名指し
`/goal 条件`	全テスト成功まで継続
`/verify`	動かして確認

立案のコツ

- 弱い依頼「テストを書いて」→ 強い依頼「ログアウト時のエッジケース。モックなし」
- テストとコードは別のClaudeに書かせると偏りにくい
- 観点の根拠（出典）を必ず付けさせる

EXAMPLES 1/9

現状分析：使用例集

すべて plan mode（読むだけ）で。範囲と出典を必ず指定する

使うコマンド

`claude --permission-mode plan`
読むだけで起動

`Shift+Tab`
plan mode on に切替

`@"code-explorer (agent)" 依頼`
調査係を確実に起動

`/context`
記憶の使用量を確認

`/btw 質問`
会話に残さず脇の質問

① 全体構成

code-explorer で、src/ の構成とエントリポイントを調べて。主要モジュール同士の呼び出し関係を、ファイルパス付きで。

② 業務フロー

請求バッチが起動してから出力までの流れを、処理順に整理して。分岐条件がある箇所はファイルと行番号を付けて。

③ 経緯を知る（出典指定）

ExecutionFactory の git 履歴を見て、このAPIがどうなってきたか要約して。関連するチケット番号が分かれば併記して。

④ 影響範囲

calcTax() の戻り値を変えたら影響する呼び出し元を全部洗い出して。テストがある／ないも分けて。

⑤ 事実・推測・不明に分ける

ここまでの調査を、確認できたこと／推測にとどまること／わからないことの3つに分けて書いて。

⑥ 脇の質問（/btw）

/btw この関数名の Impl は命名規則として何を意味している？
※ 会話の履歴に残りません

実装：使用例集

計画 → 小さな単位 → 各単位でテスト。範囲外は触らない

使うコマンド

`/plan 依頼`
実装前に計画だけ作る

`claude -w 名前`
専用worktreeで作業

`/simplify [対象]`
整理（バグ探しではない）

`/batch <指示>`
大量ファイルへ同じ変更

`/subtask <task>`
脇の作業を背景で実施

① 計画から入る

SPEC.md を読んで、請求バッチのリファクタ計画を /plan で作って。変更するファイルと順序、各段階のテスト方法を書いて。

② 小さく実装

PLAN.md の単位1だけ実装して。終わったらテストを実行し、結果を見せて。他の単位はまだ触らないで。

③ 既存を再利用

新しく書く前に、既存のユーティリティで使えるものがないか調べて。あれば @ファイルを指定して、それを使う形で実装して。

④ 一括変換（/batch）

/batch migrate src/ from JavaScript to TypeScript
※ 5〜30単位に分割→承認後に並列実装

⑤ 並行して脇の作業

/subtask 現在の変更に対するパーサーの単体テストを下書きして
※ 完了したらこの会話に戻る

⑥ 範囲外を見つけたとき

範囲外の不具合を見つけても修正しないで。SPEC.md の「やらないこと」に、場所と内容を追記だけして。

エラー対応：使用例集

「直して」の前に「再現」。2回直らなければ /clear して最初から

使うコマンド

/btw 質問
原因の脇の質問

/goal 条件
直るまで継続

/verify
実際に動かして確認

Esc Esc ／ /rewind
一つ前に戻す

/clear
学びを入れて最初から

claude --bg "依頼"
背景で調査を開始

① 再現テストが先

セッションタイムアウト後にログインが失敗する。
src/auth/ のトークン更新を見て。まず再現する
失敗テストを書き、それから直して。

② 原因を事実と推測に分ける

このエラーの原因候補を挙げて。コードで
確認できたものと推測を分け、推測には、
確かめる方法も書いて。

③ 直るまで回す

/goal `npm test` の結果が会話に表示され、
全テストが成功していること

④ 不安定なテスト（—bg）

claude --bg "flaky なテストを調べて"
→ claude agents で状況を確認
→ claude logs <id> で最近の出力

⑤ 2回直らなかったら

/clear して、ここまでで分かった原因と
試したことを書き込んだ新しいプロンプトで
最初からやり直す。

⑥ 動かして確かめる

直したあと /verify を実行して、実際に
起動して該当画面の動作を確認して。
「成功した」だけでなく操作結果を見せて。

差分確認：使用例集

AIの変更は、自分の目で差分を確認してから受け入れる

使うコマンド

`/diff`
Claudeの変更内容を確認

`/code-review [--comment]`
差分のバグ確認／PRへコメント

`git diff / git diff --staged`
未ステージ／ステージ済み（git）

`git log -p ファイル`
履歴と差分（git）

`VSCode Ctrl+Shift+G`
ソース管理で差分を見る

① 範囲外の変更がないか

今回の変更で、PLAN.md の対象外のファイルを触っていないか確認して。あれば一覧にして、理由も説明して。

② 変更の意図を説明させる

この差分を、ファイルごとに「何を・なぜ変えたか」3行以内で説明して。理由が説明できない変更は、元に戻す候補として挙げて。

③ 要件と照合

サブエージェントで、この差分を PLAN.md と照らしてレビューして。実装漏れとテスト不足のある箇所だけ報告して。

④ 旧挙動との差

リファクタ前後で出力が変わっていないか、現行テストの結果と合わせて確認して。変わった箇所は理由つきで報告して。

⑤ コミット前チェック

git diff --staged の内容を見て、デバッグ用の出力や不要なコメントが残っていないか確認して。

⑥ PRにコメント

/code-review 1234 --comment
/code-review high --fix
※ --fix は修正を適用。差分を再確認する

EXAMPLES 5/9

現在の実装から「仕様書」を作る

コードが唯一の真実なとき。読み取れたことと業務ルールの質問を分ける

使うコマンド

`claude --permission-mode plan`
読むだけで起動

`@"code-explorer (agent)"` 依頼
調査係で事実を集める

インタビュー方式
不明な業務ルールだけ質問

`SPEC.md`
書き出し先

`/clear`
書き出し後に新セッションへ

① 機能単位で逆起こし

code-explorer で @src/billing/ を調べて、
機能ごとに「入力・処理・出力」を整理し、
SPEC.md の下書きにして。行番号を付けて。

② 事実と推測を分ける

SPEC.md の各項目に、コードで確認できた(事実)／
推測／不明 のラベルを付けて。推測と不明は
末尾の「未確認事項」に集めて。

③ 不明点をインタビュー

未確認事項について、私にインタビューして。
1回1問、選択肢つきで。聞き終えたら
SPEC.md を更新して。

④ やらないことを明記

SPEC.md に「やらないこと」の節を作り、
今回の対象外の機能・画面・バッチを
理由つきで書いて。

⑤ 検証手順を末尾に

SPEC.md の最後に、エンドツーエンドの検証手順
(実行コマンドと期待結果)を書いて。
実際に動かして確認してから確定して。

⑥ 仕様書で新セッション開始

/clear して SPEC.md を読んで。その仕様に
沿って、請求バッチの現行動作を固定する
テスト計画を /plan で作って。

EXAMPLES 6/9

現在の実装から「要件書」を作る

「コードがそうなっている」と「そうあるべき」は別。確認を挟んで要件にする

使うコマンド

`plan mode`
読むだけで調査

`code-explorer`
根拠つきで事実収集

インタビュー方式
要件の意図を確認

`REQUIREMENTS.md`
書き出し先（名前は自由）

`/btw 質問`
脇の確認は履歴に残さない

① 現行動作を要件候補に

code-explorer で現行コードの振る舞いを調べ、
要件候補を「〜できること」の形で列挙して。
根拠のファイルと行を付けて。

② 機能／非機能に分類

要件候補を、機能要件／非機能要件（性能・
セキュリティ・運用）／業務ルール に分類して。
分類が不明なものは別枠にして。

③ バグ疑いを分離

現行コードの挙動で、仕様として意図したのか
バグか判断できないものを一覧にして。
私に質問して、要件に含めるか決める。

④ 受入条件を付ける

各要件に、実行できる受入条件を付けて。
例：入力Aのとき出力Bになる。テストで
確認できる形に。

⑤ 優先度と範囲

要件を、必須／あると良い／対象外に分けて。
対象外は理由を書き、SPEC.md の
「やらないこと」と整合させて。

⑥ 食い違いの確認

REQUIREMENTS.md と SPEC.md を照らして、
食い違い・抜けを報告して。スタイルの好みは
不要。不足だけ。

EXAMPLES 7/9

禁止・制約を操るコマンド

「お願い」ではなく「仕組み」で縛る。弱い順：依頼文 → CLAUDE.md → 権限・ツール → フック

使うコマンド

`--permission-mode plan`
ファイルを変更させない

`--permission-mode dontAsk`
承認が要る操作を拒否（放置実行）

`--allowedTools "Edit,Bash(git commit *)"`
必要なツールだけ事前承認

`tools / disallowedTools`
サブエージェントの許可・拒否

`/hooks`
Stopフック等の決定論的な門番

`/memory`
CLAUDE.md に禁止事項を追記

`/permissions`
allow/ask/denyルールを管理

① 読み取り専用の調査係

name: code-explorer／tools: Read, Glob, Grep
本文に「Never modify files」。
→ 書き込み系ツールを渡さない

② 本番コード変更を禁止

test-writer の本文に「Do not change production code」と書き、isolation: worktree で隔離。
変更が出たら /diff で確認する

③ 放置実行を安全に

claude -p "…" --allowedTools "Edit,Bash(git commit *)" --permission-mode dontAsk
→ 許可外の操作は自動で拒否

④ 禁止事項を常時ルール化

/memory で CLAUDE.md に追記：「DBスキーマは変更しない。.env は読まない」。短く、
消したらAIが間違うものだけ

⑤ テスト未通過を許さない

Stopフック：テストが通るまでターン終了をブロック。/hooks で設定。
「お願い」ではなく決定論で止める

⑥ 禁止の例外を許可する

範囲を一時的に広げるときは SPEC.md の「やらないこと」を更新してから依頼する。
口頭の例外は次のセッションで消える

EXAMPLES 8/9

サブエージェントの使用例集

定義は .claude/agents/*.md（name と description が必須）。description は「いつ任せるか」

使うコマンド

@"名前 (agent)" 依頼
確実に起動（@メンション）

claude --agent 名前
セッション全体を1役に

Ctrl+B
実行中を背景へ

background: true
常に背景で動かす

isolation: worktree
隔離して実行

model / maxTurns / effort
モデル・ターン上限・思考量

① 定義：調べる人

name: code-explorer／tools: Read, Glob, Grep
model: sonnet／本文：「事実・推測・
未解決の質問」の3点で報告する

② 3人チームで進める

code-explorer で現状→test-writer で現行テスト→
reviewer で確認、の順で請求バッチの改修を
進めて。各段階の結果を要約して。

③ 指名して起動

@"code-explorer (agent)" 認証の変更点を見て
※ 自然な言葉より確実に起動できる

④ 並列の上限に注意

同時に動くサブエージェントは既定で最大20。
入れ子は3階層まで。1で入れ子を無効化
（環境変数で変更可）

⑤ レビュー役の頼み方

Use a subagent to review the diff against
PLAN.md…Report gaps, not style preferences.
→ 不足だけ報告させる

⑥ 置き場所と優先度

管理設定＞--agents＞.claude/agents/＞
~/.claude/agents/＞プラグイン。
名前が衝突したら優先度の高い方が勝つ

EXAMPLES 9/9

Skill編集：使用例集

SKILL.md は依頼文かVSCodeで直接編集。/skills で一覧、/reload-skills で再読込（公式commandsで確認）

使うコマンド

`/skills`
スキル一覧（名前・説明で絞込）

`/reload-skills`
編集したスキルを再読込

`/skill-doctor`
コスト・利用頻度を確認

`/run-skill-generator`
起動方法のスキルを作る

VSCodeで `SKILL.md` を開く
直接編集（Ctrl+P）

`/diff`
編集内容の差分確認

① 手順をスキルに格上げ

今日の調査手順を、.claude/skills/investigate-billing/SKILL.md にして。description は「請求バッチ調査の依頼を受けたとき」。

② description を直す

SKILL.md の description を、いつ使うか具体的になるよう書き直して。「〇〇と言われたとき」「△△のファイルを触るとき」の形で。

③ 長いスキルを削る

SKILL.md を見直して、コードを読めば分かる説明とAPIの詳細は削除し、詳細はリンクにして。短くして。

④「やらないこと」を追加

SKILL.md の末尾に「やらないこと」を追加して：本番コードは変更しない／範囲外は報告のみ。

⑤ 効いているか試す

/clear して、スキルの対象になる依頼を出して。スキルの手順どおりに動くか確認し、外れた箇所を SKILL.md に反映して。

⑥ スキルかフックか迷ったら

毎回必ず実行したい → フック。
たまに使う手順・知識 → スキル。
常に守るルール → CLAUDE.md。

SCENES | MAP

シーン別コマンド：3編の地図

公式ドキュメント（英語版・日本語版）を参考。日本語版は英語版より更新が遅れることがあるので、食い違ったら英語版を優先

基本編　BASIC

- 初回セットアップ
- 作業中の操作
- 戻す・見る・再開
- モデル・設定
- VSCodeでの使い方

毎日使う。まずここだけ覚える

応用編　ADVANCED

- 並列・分割
- 品質確認・レビュー
- 権限・設定の自動化
- スキル・サブエージェント
- 定期実行（/loop）

チーム開発・引き継ぎ案件で効く

ニッチ編　NICHE

- 利用分析・オンボーディング
- 他ツールからの移行
- クラウド・別端末
- サンドボックス・隔離
- 表示・入力の微調整

知っていると差がつく

公式の入口（英語）https://code.claude.com/docs/en/commands　／　（日本語）https://code.claude.com/docs/ja/commands
全ページ索引：https://code.claude.com/docs/llms.txt　|　一覧は Claude Code 内で / を入力、最新の変更は /release-notes

基本編：シーン別コマンド

毎日使う。VSCodeの統合ターミナルでもそのまま使えます

シーン	コマンド	何をする	一言例
初めてのリポジトリ	`/init → /memory`	CLAUDE.md を作って育てる	「このプロジェクトの規約を CLAUDE.md に」
大きな変更の前	`/plan 依頼`	プランモードで計画だけ立てる	/plan fix the auth bug
モデル・思考量	`/model /effort`	切り替え。effort は low〜xhigh,max,auto	/effort high
会話が長い	`/context → /compact`	使用量を見て、要約して空ける	/compact 要件の話だけ残して
脇の質問	`/btw 質問`	会話の履歴に足さず質問	/btw この Impl の意味は？
差分・戻す	`/diff /rewind`	変更確認／コードと会話を巻き戻す（別名 /undo）	作業前にコミットも併用
続き・新規	`/resume /clear /branch`	再開／空にする／分岐して別案を試す	/clear [名前] で前の会話に名前
VSCode	`/terminal-setup /ide`	Shift+Enter 改行の設定／IDE連携の管理	/bug は拡張側のフィードバック画面

基本編：使用例と公式URL

英語版が最新。日本語版（ja）があるページは言語部分を差し替え

公式URL（基本編）

Commands（英）	https://code.claude.com/docs/en/commands
コマンド（日）	https://code.claude.com/docs/ja/commands
Best practices（英）	https://code.claude.com/docs/en/best-practices
ベストプラクティス（日）	https://code.claude.com/docs/ja/best-practices
CLI reference（英）	https://code.claude.com/docs/en/cli-reference
CLIリファレンス（日）	https://code.claude.com/docs/ja/cli-reference
Interactive mode（英）	https://code.claude.com/docs/en/interactive-mode
Checkpointing（英）	https://code.claude.com/docs/en/checkpointing
VS Code（英）	https://code.claude.com/docs/en/vs-code

例：初日の立ち上げ

/init → /memory で CLAUDE.md を短く整える
/plan 請求バッチの構造を調べて計画を作って

例：会話が重いとき

/context で何が容量を使っているか確認
/compact エラーの原因と決定事項だけ残して

例：やり直したいとき

/rewind でチェックポイントまで戻す
別方針は /branch 案B で分岐して試す
元の会話は /resume で戻れる

応用編：シーン別コマンド

チーム開発・引き継ぎ案件向け。バンドルスキル（/batch /verify など）はプロンプトで動くスキルです

シーン	コマンド	何をする	一言例
大量ファイルを変える	`/batch 指示`	5〜30単位に分け、worktreeで並列実装	/batch migrate src/ to TypeScript
脇の作業・並走	`/subtask /fork /background`	サブエージェント／別セッション／背景へ	/subtask パーサーのテストを下書き
完了まで回す	`/goal 条件`	条件を満たすまで継続（clearで解除）	/goal `npm test` が全部成功
動かして確認	`/run /verify`	アプリを起動して変更を観察	/verify 請求画面の変更
レビュー	`/code-review /security-review /simplify`	バグ／セキュリティ／整理（別名 /review）	/code-review high --fix
権限・設定	`/permissions /update-config /fewer-permission-prompts`	ルール管理／settings.json編集／許可リスト提案	「git commit を許可して」
スキル・エージェント	`/skills /reload-skills /agents /list-agents`	一覧・再読込・サブエージェント案内・宛先一覧	編集後は /reload-skills
診断・定期実行	`/doctor /debug /loop /schedule`	セットアップ診断／ログ解析／繰り返し／クラウド定期	/loop 5m デプロイ完了を確認

応用編：使用例と公式URL

権限・フック・スキルは「仕組みで縛る・仕組みで回す」ための公式ページ

公式URL（応用編）

Skills（英）	https://code.claude.com/docs/en/skills
スキル（日）	https://code.claude.com/docs/ja/skills
Subagents（英）	https://code.claude.com/docs/en/sub-agents
Permissions（英）	https://code.claude.com/docs/en/permissions
権限モード（日）	https://code.claude.com/docs/ja/permission-modes
Hooks（英）	https://code.claude.com/docs/en/hooks
Worktrees（英）	https://code.claude.com/docs/en/worktrees
Agent view（英）	https://code.claude.com/docs/en/agent-view
/goal（日）	https://code.claude.com/docs/ja/goal
Local code review（英）	https://code.claude.com/docs/en/code-review-local

例：許可を減らす

/fewer-permission-prompts で読み取り系の
許可リストを提案させ、project の
settings.json に追加

例：スキル編集後

SKILL.md を直したら /reload-skills
/skills で一覧に出るか確認
/skill-doctor でコストを見る

例：引き継ぎ案件の締め

/simplify で整理 → /code-review high --fix
→ /security-review → /verify
最後に別セッションで要件と照合

ニッチ編：シーン別コマンド

プラン・環境によって表示されないコマンドがあります（公式の注記）。使えるかは / のメニューで確認

シーン	コマンド	何をする	一言例
使い方の振り返り	`/insights` `/team-onboarding`	利用レポート／チーム向けオンボーディング資料	過去30日の履歴から生成
他ツールから移行	`/import codex|gemini|cursor`	指示・MCP・スキル等を取り込む（--dry-runで確認）	/import cursor --dry-run
PRを自動で直す	`/autofix-pr` `/code-review ultra`	CI失敗・コメントに対応／クラウドで深いレビュー	/autofix-pr lintと型エラーだけ
別端末・クラウド	`/teleport` `/remote-control` `/desktop`	クラウドセッション取得／別端末から継続／デスクトップへ	自席→外出先で続き
隔離・範囲	`/sandbox` `/add-dir` `/cd`	サンドボックス／追加ディレクトリ／作業場所の移動	別リポジトリを参照
調査・複数セッション	`/deep-research` `/list-agents` `/workflows`	出典つき調査／宛先一覧／ワークフロー進行表示	/deep-research 比較したい論点
コスト・長さ	`/usage` `/autocompact` `/advisor`	使用状況／自動圧縮の幅／助言モデル	/autocompact 500k
表示・保存	`/copy` `/export` `/recap` `/focus` `/statusline`	応答のコピー／書き出し／要約／集中表示／ステータス行	/copy 2 で2つ前の応答

ニッチ編：使用例と公式URL

一部は実験的・バージョン要件あり。使う前に公式で現状を確認

公式URL（ニッチ編）

Agent teams（英）	https://code.claude.com/docs/en/agent-teams
Cross-session（英）	https://code.claude.com/docs/en/cross-session-messaging
Sandboxing（英）	https://code.claude.com/docs/en/sandboxing
Routines（英）	https://code.claude.com/docs/en/routines
Ultrareview（英）	https://code.claude.com/docs/en/ultrareview
Workflows（英）	https://code.claude.com/docs/en/workflows
Scheduled tasks（英）	https://code.claude.com/docs/en/scheduled-tasks
.claude directory（英）	https://code.claude.com/docs/en/claude-directory
全ページ索引	https://code.claude.com/docs/llms.txt

例：他ツールの設定を持ち込む

/import cursor --dry-run で内容を確認
問題なければ --yes を付けて取り込む

例：使い方を棚卸し

/insights で直近の利用レポートを作成
/team-onboarding で新メンバー向けの
最初のメッセージ用ガイドを作る

例：定期チェックを任せる

/loop 5m デプロイが終わったか確認して
クラウドで回すなら /schedule（ルーティン）
止めたいときは /goal clear 等で解除

Docker Desktop 連携：MCP Toolkit 経由

Docker Desktop の MCP Toolkit が「ゲートウェイ」になり、Claude Code は1つのMCPサーバーとして接続します

1 Toolkit を有効化

Docker Desktop で MCP Toolkit を有効にする

2 サーバーを追加

MCP Toolkit の Catalog タブからサーバー（例：Docker Hub）を追加

3 Claude Code を接続

Clients タブ、またはCLIで接続

4 確認

claude mcp list ／ /mcp で Connected を確認

CLIでの接続（どちらか）

```
# 方法A：Docker側から接続
docker mcp client connect claude-code

# 方法B：Claude Code側から登録（user スコープ）
claude mcp add MCP_DOCKER -s user -- docker mcp gateway run

# 確認
claude mcp list
# → MCP_DOCKER: docker mcp gateway run - ✓ Connected

# プロファイルを指定して起動する場合
docker mcp gateway run --profile my_profile
```

ポイント

- Claude Code からは MCP_DOCKER という1つのサーバーに見える
- サーバーの追加・削除は Docker Desktop 側で行う
- 接続後に Claude Code を再起動 → /mcp で確認
- 認証が要るサーバー（Docker Hub等）は Docker側でシークレット設定
- 公式ガイドでは起動コマンドを claude code と書いているが、通常は claude

Docker 連携：使用例・トラブル・公式URL

Docker Desktop が通信を横取りしてエラーになることも。まず終了して切り分け（公式のエラー参照）

例：Compose を自然言語で作る

Docker Hub のMCPで Node.js と PostgreSQL の
docker-compose.yml を作って保存して。
→ 保存後 docker compose up で起動

例：接続されているか確認

claude mcp list で MCP_DOCKER を確認
Claude Code 内で /mcp → ツール数を確認
接続が切れたら /mcp reconnect all

例：チームで共有

claude mcp add --scope project … で
全員同じ設定に。.mcp.json をコミットすると
初回に各自が承認する形になる

トラブル切り分け

- Unable to connect to API → Docker Desktop などコンテナランタイムが外向き通信を横取りしていないか。終了して再試行
- MCP_DOCKER が Failed to connect → docker mcp gateway run を単体で実行して出力確認
- Needs authentication → /mcp で認証、または claude mcp login 名前
- ツールが多すぎて重い → /context、不要なサーバーを /mcp で無効化

公式URL（Docker）

MCP Toolkit 入門	https://docs.docker.com/ai/mcp-catalog-and-toolkit/get-started/
Claude Code ガイド	https://docs.docker.com/guides/genai-claude-code-mcp/
Docker公式ブログ	https://www.docker.com/blog/add-mcp-servers-to-claude-code-with-mcp-toolkit/
Claude Code MCP（英）	https://code.claude.com/docs/en/mcp

MCP：追加・管理コマンド

公式 Connect Claude Code to tools via MCP に基づく。HTTP が推奨、SSE は非推奨

やりたいこと	コマンド	補足	例
リモートを追加(HTTP)	`claude mcp add --transport http 名前 URL`	推奨のトランスポート	… notion https://mcp.notion.com/mcp
ローカルを追加(stdio)	`claude mcp add 名前 -- コマンド 引数`	-- 以降はサーバーにそのまま渡る	… db -- npx -y @bytebase/dbhub --dsn …
JSONから追加	`claude mcp add-json 名前 '{…}'`	wsはこちら。--scope も使える	… events '{"type":"ws",…}'
スコープ指定	`--scope local|project|user`	local=自分・このPJ／project=.mcp.json／user=全PJ	--scope project（チーム共有）
一覧・詳細・削除	`claude mcp list / get 名前 / remove 名前`	状態は ✔ Connected 等で表示	claude mcp get github
認証	`/mcp  claude mcp login 名前`	OAuth。logout で解除	claude mcp login sentry
他から取り込む	`claude mcp add-from-claude-desktop`	macOS と WSL のみ	取り込みサーバーを選択
承認リセット	`claude mcp reset-project-choices`	.mcp.json の承認をやり直す	初回承認を選び直す

MCP 連携：使用例集

DX案件で効く3系統：課題管理・DB・GitHub。認証情報は .mcp.json に直書きしない（${VAR}展開を使う）

使うコマンド

`claude mcp add`
サーバー追加

`claude mcp list`
状態確認

`/mcp`
パネルで認証・再接続・無効化

`@server:protocol://path`
MCPリソースを参照

`--mcp-config`
指定ファイルのMCPだけで起動

`--strict-mcp-config`
上記以外のMCPを使わない

① GitHub でレビュー

claude mcp add --transport http github
https://api.githubcopilot.com/mcp/ --header
"Authorization: Bearer <PAT>"
→「PR #456 をレビューして」

② DBを読み取り専用で

claude mcp add --transport stdio db --
npx -y @bytebase/dbhub --dsn
"postgresql://readonly:…"
→ readonly ユーザーで誤更新を防ぐ

③ 課題管理から実装

claude mcp add --transport http notion
https://mcp.notion.com/mcp
→「ENG-4521 の仕様を読んで実装、PRまで」

④ チーム共有（.mcp.json）

claude mcp add --transport http --scope project
shared-server https://example.com/mcp
→ コミット。初回は各自が承認

⑤ 環境変数で秘密を分離

"url": "${API_BASE_URL:-https://api.example.com}/mcp"
"headers": {"Authorization": "Bearer ${API_KEY}"}
※ ANTHROPIC_AUTH_TOKEN 等は空として扱われる

⑥ リソースを参照

@github:issue://123 を分析して修正案を出して
@postgres:schema://users と @docs:file://…
を比較して

MCP 連携のエラー対応

まず claude mcp list で状態、次に claude mcp get 名前 の Issue 行を見る

症状・表示	原因の目安	対処コマンド
✘ Failed to connect	起動失敗・URL違い・認証情報不備	claude mcp get 名前 → Issue 行／ MCP_TIMEOUT=10000 claude
! Needs authentication	401/403・OAuth未了	/mcp で認証、または claude mcp login 名前
Pending approval	project の .mcp.json が未承認	claude を対話で起動して承認／ claude mcp reset-project-choices
has a "url" but no "type"	JSONに type がない（stdio扱い）	"type": "http" を追加（sse/ws も可）
spawn claude ENOENT	claude が PATH にない	which claude → フルパスを command に指定
MCP server "all" not found	古い版で reconnect all を実行	v2.1.284 以降へ更新（claude update）
同名が複数スコープ	別のエンドポイントで警告	claude mcp remove 名前 --scope <scope>
出力が大きい／遅い	出力上限・タイムアウト	MAX_MCP_OUTPUT_TOKENS=50000／ per-server "timeout"／ /tasks

切断時：リモートは最大5回まで自動再接続。失敗後は /mcp から手動、または /mcp reconnect all。stdio は自動再接続しません。

エラー対応コマンド：サーバー・利用上限・認証

公式 Error reference に基づく。大半は自動で最大10回まで再試行された後に表示される

表示されるメッセージ	意味	やること
API Error: 500 / Repeated 529	API側の障害・混雑（あなたの上限ではない）	status.claude.com を確認 → 数分待つ／ /model で別モデル／ /feedback
Request timed out	10分で応答なし	分割して再試行／ API_TIMEOUT_MS を延ばす
…response above may be incomplete	途中で切れた	「continue」と返信（工具の二重実行を避けるため再送されない）
You've hit your session limit	プランの利用上限	/usage で確認／ /usage-credits／ Opus上限なら /model で切替
Request rejected (429)	APIキー等のレート制限	/status で認証確認／ CLAUDE_CODE_MAX_TOOL_USE_CONCURRENCY を下げる
Not logged in · Please run /login	認証がない	/login ／ ANTHROPIC_API_KEY が export されているか確認
Invalid API key	キーが無効・古い	env | grep ANTHROPIC ／ unset して /login ／ /status
Login expired / OAuth token expired	保存ログインが失効	/login（直らなければ /logout → /login）

エラー対応コマンド：リクエスト・ネットワーク・モデル

Docker Desktop などコンテナランタイムが通信を横取りして Unable to connect になることもある

表示されるメッセージ	意味	やること
Prompt is too long	コンテキスト超過	/compact か /clear／ /context で内訳／ 使わない MCP を /mcp で無効化
Error during compaction: Conversation too long	`/compact` 自体が失敗	Esc Esc で数ターン戻して再度 /compact／ だめなら /clear
Request too large / Image was too large	本体サイズ・画像サイズ超過	Esc Esc で戻る／ 大きなファイルはパス指定／ 画像は縮小
API Error: 400 … tool use concurrency	`tool_use` の履歴が不整合	/rewind で壊れた回の前へ（Opus 4.7/4.8 は claude update）
Unable to connect to API	接続不可・VPN・プロキシ	curl -I https://api.anthropic.com／ HTTPS_PROXY ／ Docker等を終了
SSL certificate verification failed	社内プロキシの証明書	NODE_EXTRA_CA_CERTS=CAバンドル／ claude doctor（TLS検証は切らない）
issue with the selected model	モデル名が無効・権限なし	/model で選び直す／ 古い ID を設定から削除（別名 sonnet/opus を推奨）
File is covered by a Read deny rule	`Read` の `deny` に該当	/permissions で見直す／ 触らせたくないなら Edit deny も追加

エラー対応：調べ方の使用例集

「再現・ログ・戻す」の順。直らなければ /feedback で文字起こしを送る

使うコマンド

/doctor
セットアップ診断（修正案も）

claude doctor
シェルから読み取り診断

/debug 説明
デバッグログを読んで原因分析

claude --debug-file /tmp/claude-debug.log
ログを指定ファイルへ

/status
使っている認証・環境を確認

/feedback
文字起こし付きで報告

① まず診断

/doctor を実行して、設定・CLAUDE.md・拡張に問題がないか確認し、直せるものは提案して。

② ログで切り分け

claude --debug-file /tmp/claude-debug.log
→ 再現 → ログで never expanded toward a remote server 等を検索
/debug で原因を要約させる

③ 認証の取り違え

/status で認証方法を確認。環境の ANTHROPIC_API_KEY がサブスクより優先されていないか。不要なら unset して /login

④ 再現してから直す

エラー全文を貼り、「まず再現する失敗テストを書いて、それから直して」と依頼。テスト出力を見せてもらう

⑤ 履歴が壊れたとき

/rewind（または Esc Esc）で壊れた回の前へ。同じ指摘を2回しても直らなければ /clear して試したことを書いて最初から

⑥ 無人実行での再試行

CI では CLAUDE_CODE_RETRY_WATCHDOG=1 で 429/529 を無期限に再試行。短くしたいなら CLAUDE_CODE_MAX_RETRIES を下げる

開発環境構築：インストール・更新・診断

公式 Advanced setup / Installation に基づく。ネイティブインストールは背景で自動更新

目的	コマンド	補足
macOS / Linux / WSL	`curl -fsSL https://claude.ai/install.sh | bash`	ネイティブ（推奨）。WSLではWSL端末の中で実行
Windows PowerShell	`irm https://claude.ai/install.ps1 | iex`	Git for Windows は推奨。無ければPowerShellツールで動作
Windows CMD	`curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del install.cmd`	同上
Homebrew / npm	`brew install --cask claude-code` ／ `npm install -g @anthropic-ai/claude-code`	npm版も同じネイティブバイナリを入れる
診断	`claude doctor`	インストール・設定・直近の更新結果を確認
更新・版指定	`claude update` ／ `claude install [version|stable|latest]`	チャンネルは autoUpdatesChannel 設定。DISABLE_AUTOUPDATER は自動確認のみ停止
認証	`claude auth login` ／ `claude auth status` ／ `/login`	CI用は claude setup-token で長期トークン
起動確認	`claude --version` ／ `claude`	初回起動でログインへ

公式（英）https://code.claude.com/docs/en/setup ／ https://code.claude.com/docs/en/getting-started/installation ／ 事前確認：Windows 10 1809+ または Windows Server 2019+

開発環境構築：VSCode・プロジェクト初期設定

統合ターミナル（Ctrl+`）で claude を起動。WSLの場合は VSCode をWSLに接続して同じ端末で実行

やりたいこと	コマンド	何をする
VSCodeと連携	`/ide`	IDE連携の管理と状態表示
Shift+Enterで改行	`/terminal-setup`	VSCode・Cursor等にキーバインドを設定
プロジェクト初期化	`/init → /memory`	CLAUDE.md を生成して整える
権限ルール	`/permissions  /update-config`	allow・ask・deny／settings.json の編集依頼
MCP接続	`/mcp  claude mcp add …`	必要なサーバーを接続（Dockerは別ページ）
別ディレクトリ	`/add-dir  /cd`	参照ディレクトリ追加／作業場所の移動
設定画面・表示	`/config  /statusline  /theme`	モデル・出力スタイル・ステータス行など
隔離して実行	`/sandbox  claude -w 名前`	サンドボックス／専用worktreeで作業

チームで共有するもの（コミット対象）：CLAUDE.md・.mcp.json・.claude/agents/・.claude/skills/・.claude/settings.json。個人の設定は ~/.claude 側へ。

開発環境構築：使用例集

「README を読んで手順を出す → 承認して実行 → /doctor で確認」の順。いきなり実行させない

使うコマンド

`claude doctor`
インストール診断

`/doctor`
設定診断（CLAUDE.md等も）

`/init`
CLAUDE.md 生成

`/permissions`
許可ルールの管理

`/mcp`
MCP接続状況

`--permission-mode plan`
まず読むだけで起動

① 手順を出させる（plan mode）

README と package.json を読んで、この環境の
構築手順を実行コマンドの順に書き出して。
まだ実行しないで。

② 承認してから実行

手順1〜3だけ実行して、各コマンドの出力を
見せて。失敗したら原因を事実と推測に分けて。

③ 環境差を洗い出す

Windows・WSL・Mac で手順が違う箇所を表にして。
パスと改行コードの違いも挙げて。

④ 設定をチームで共有

今の MCP と権限ルールを .mcp.json と
settings.json にまとめて。秘密は ${VAR} 参照にして。

⑤ 検証する

/doctor を実行して、問題と修正案を出して。
直したら /status で認証と環境を確認。

⑥ 手順書にして残す

今日の構築手順を SKILL.md にして .claude/skills/
setup-env/ へ。description は「環境構築を頼まれたとき」

開発環境構築のエラー対応

インストールとログインのエラーは公式 Troubleshoot installation and login も参照

表示・症状	原因の目安	対処
Installation was killed … (exit code 137)	メモリ不足（OOM）。目安 約512MB	他の処理を止める／スワップ追加 → 再実行
The connection dropped while downloading…	プロキシが大きい転送を切る	claude update を再実行／ HTTPS_PROXY ／ downloads.claude.ai を許可
Download timed out: exceeded the total deadline	10分以内に完了しない	より速い回線で再実行（再試行されない）
SSL certificate verification failed	社内プロキシの TLS検査	NODE_EXTRA_CA_CERTS=CAバンドル／ claude doctor
WSL で claude が動かない	PowerShell/CMD から起動している	WSL端末の中でLinux用インストーラを実行して起動
Alpine 等でインストール失敗	musl 系に必要なパッケージ不足	libgcc・libstdc++・ripgrep を入れる
npm版が自動更新されない	npmのグローバル先に書込不可	claude doctor の案内に従う／ネイティブ版へ移行
Not logged in · Please run /login	認証がない	/login ／ claude auth status ／ ANTHROPIC_API_KEY を確認

公式（英）https://code.claude.com/docs/en/troubleshoot-install ／ エラー参照：https://code.claude.com/docs/zh-TW/errors（取得できた版）

Skill 管理コマンド

公式 Commands と Skills に基づく。カスタムコマンドはスキルに統合済み（.claude/commands/ も動作）

やりたいこと	コマンド	何をする
一覧を見る	`/skills`	利用可能なスキルを一覧・絞り込み。t でトークン順
編集を反映	`/reload-skills`	ディスク上の追加・変更を再スキャン（再起動不要）
コストと使用頻度	`/skill-doctor`	文脈コストと使用回数を確認し、不要なスキルを見つける
手動で呼ぶ	**`/スキル名　引数`**	最大6つまで連結：/skill-a /skill-b 依頼
起動方法を覚えさせる	`/run-skill-generator`	/run と /verify 用のプロジェクト別スキルを作る
拡張・設定の診断	`/doctor　/update-config`	CLAUDE.md・拡張の診断／settings.json の編集依頼
バンドル済み	`/batch　/loop　/simplify　/debug　/verify`	プロンプトで動くスキル。中身を確認して調整できる
置き場所	**`~/.claude/skills/ ／ .claude/skills/ ／ プラグイン`**	個人／プロジェクト／配布

公式 https://code.claude.com/docs/en/skills　／　（日）https://code.claude.com/docs/ja/skills　※ 自動で呼ばれたくないスキルは frontmatter の disable-model-invocation: true

Skill：使用例集

手順が文章に育ったら CLAUDE.md からスキルへ。長い資料は必要になるまで読み込まれない

使うコマンド

/skills
一覧・並べ替え

/reload-skills
編集の反映

/skill-doctor
コスト確認

/スキル名
手動呼び出し

/diff
編集差分の確認

/clear
新セッションで試す

① 手順をスキルにする

今日の請求バッチ調査の手順を .claude/skills/billing-investigate/SKILL.md にして。description は「請求バッチの調査を頼まれたとき」。

② 重いスキルを見つける

/skill-doctor で文脈コストが高い・使われていないスキルを確認。不要なら無効化か削除を相談。

③ 編集して反映

SKILL.md の手順2を直して。保存後 /reload-skills、/skills で一覧に出るか確認してから試す。

④ 連結して使う

/review-checklist /security-notes この差分をチェックして。（最大6つまで連結）

⑤ 勝手に動かしたくない

frontmatter に disable-model-invocation: true。呼ぶときは自分で /スキル名 と入力。

⑥ サブエージェントに読ませる

サブエージェントの skills: に指定すると起動時に全文が注入される（継承はされない）。

サブエージェント：起動・管理コマンド

公式 Create custom subagents に基づく。定義は Markdown＋YAML、必須は name と description

やりたいこと	コマンド・書き方	補足
定義する	`~/.claude/agents/ 名.md ／ .claude/agents/ 名.md`	個人／プロジェクト。編集は数秒で反映（新規ディレクトリは再起動）
確実に起動	`@"名前 (agent)" 依頼`	@メンション。自然言語は Claude が判断
全体を1役にする	`claude --agent 名前`	プロンプトも置き換わる。setting の agent で既定にも
その場限りで定義	`claude --agents '{"名前":{"description":…,"prompt":…}}'`	セッション限定。-p ならJSONファイルのパスも可（v2.1.281〜）
状況を見る	`/tasks  Ctrl+B`	背景作業・完了済みの確認／実行中を背景へ
会話を引き継ぐ	`/subtask 依頼  /fork [依頼]`	フォーク型サブエージェント／別の背景セッション
再開・追加指示	「前の作業を続けて」	Claude が SendMessage で再開。IDは ~/.claude/projects/… に保存
定義の検証	`claude plugin validate .claude/agents`	YAML が壊れたファイルを事前検出（v2.1.233〜）

公式（英）https://code.claude.com/docs/en/sub-agents ／ 関連：agent-view・agent-teams・cross-session-messaging・worktrees（いずれも /docs/en/ 配下）

サブエージェント：frontmatter 早見表

綴りは camelCase で表と完全一致。認識できない項目は黙って無視される

項目	役割	例・注意
tools / disallowedTools	許可／拒否するツール	tools: Read, Grep, Glob ／ disallowedTools: mcp_github
model	使うモデル	sonnet・opus・haiku・fable・フルID・inherit
permissionMode	権限モード	default・acceptEdits・auto・dontAsk・bypassPermissions・plan
maxTurns	ターン上限	超えたら部分結果として返り、再開できる
skills	起動時に全文を注入	親のスキルは継承されない
mcpServers	このエージェント専用の MCP	インラインで定義すると親の文脈を汚さない
memory / background	永続メモリ／常に背景	memory: user・project・local
isolation / effort / omitClaudeMd	隔離・思考量・ CLAUDE.md省略	isolation: worktree ／ effort: low〜max

優先順位：管理設定 ＞ --agents ＞ .claude/agents/ ＞ ~/.claude/agents/ ＞ プラグイン。プラグイン配布の定義では hooks・mcpServers・permissionMode は無視される。

サブエージェント：使用例集

「狭いツール」「別の文脈」「隔離」が三本柱。1体1役にする

使うコマンド

`@"名前 (agent)" 依頼`
確実に起動

`claude --agent 名前`
全体を1役に

`/subtask 依頼`
文脈ごと引き継ぐ

`/tasks`
状況・モデル確認

`Ctrl+B`
背景へ

`--disallowedTools "Agent(Explore)"`
特定の組み込みを無効化

① 読み取り専用レビュー役

name: code-reviewer ／ tools: Read, Glob, Grep
model: sonnet。本文に「変更しない・不足だけ報告」

② 並列で調査

認証・DB・APIの3モジュールを別々のサブエージェントで
並列に調べて。結果は事実／推測／不明で要約して。

③ 連鎖させる

code-reviewer で性能の問題を見つけ、その結果を
optimizer に渡して直して。各段階の要約を出して。

④ 重い出力を隔離

サブエージェントでテスト全体を実行し、
失敗したテストとエラーだけ報告して。

⑤ 本番を触らせない

test-writer に isolation: worktree を付け、
「本番コードは変更しない」と本文に明記。

⑥ コストを抑える

調査役は model: haiku、設計役は opus など
役割でモデルを分ける。全員を揃えるなら
CLAUDE_CODE_SUBAGENT_MODEL_FORCE=1

サブエージェントのエラー・制限

表示されるメッセージは公式 Error reference と Subagents ページに基づく

表示・症状	原因の目安	対処
Agent would be spawned with zero tools	`tools` の綴り違い・存在しないツール	tools を直す／ tools 自体を消して全継承
Concurrent subagent limit reached	同時実行が既定の 20に到達	完了を待つ／ CLAUDE_CODE_MAX_CONCURRENT_SUBAGENTS
Agent terminated early due to an API error	利用上限・サーバーエラー	原因を解消 → Claude に再試行か再開を依頼
subagent_type is required	`general-purpose` が無い環境	型を指定して呼ぶ／ 型を明示するよう依頼
Invalid —agents configuration	`--agents` のJSONが不正	JSONと項目名を直す（prompt・description など）
エージェントが見つからない	`name` が無い／ YAML不正／ `description` なし	claude --debug のログ／ claude plugin validate
新しい agents フォルダが反映されない	セッション開始時に無かったディレクトリ	Claude Code を再起動
説明の合計が15,000トークン超の警告	`description` が長すぎる	description を短く、詳細は本文（システムプロンプト）へ

入れ子は既定で3階層まで（CLAUDE_CODE_MAX_SUBAGENT_SPAWN_DEPTH で変更、1で無効）。同じ name が複数あると /doctor が一覧して整理案を出します。

RECOVERY

戻し方・差分確認・新しく始める

失敗しても戻れるように、作業の前にコミットしておく

戻し方

`Esc Esc ／ /rewind`	一つ前の時点に戻す
`/clear`	会話を空にする（文書は残る）
`git restore ファイル`	変更を元に戻す（git）
`git stash`	作業を一時退避（git）
`git switch -`	直前のブランチへ（git）

差分確認

`/diff`	Claudeの変更内容を確認
`git diff`	未ステージの差分（git）
`git diff --staged`	ステージ済みの差分（git）
`git log -p ファイル`	ファイルの履歴と差分（git）
`VSCode ソース管理`	Ctrl+Shift+G でファイルを選ぶ

新しく始める

`/clear`	無関係な作業の間に
`/branch 名前`	会話を分岐して別方針を試す
`/fork プロンプト`	会話をコピーして別セッションへ
`claude -w 名前`	新しいworktreeで始める
`claude --bg "依頼"`	背景で新しく開始

目安：同じ指摘を2回しても直らないなら /clear して、学んだことを入れたプロンプトで最初から。戻し方の（git）は一般的なgitコマンドで、Claude Code固有ではありません。/rewind はコードと会話をチェックポイントまで戻します（別名 /undo）。